

Module 7: Signing and Cryptography

Ed25519 Digital Signatures & Key Management

Cryptographic Foundations

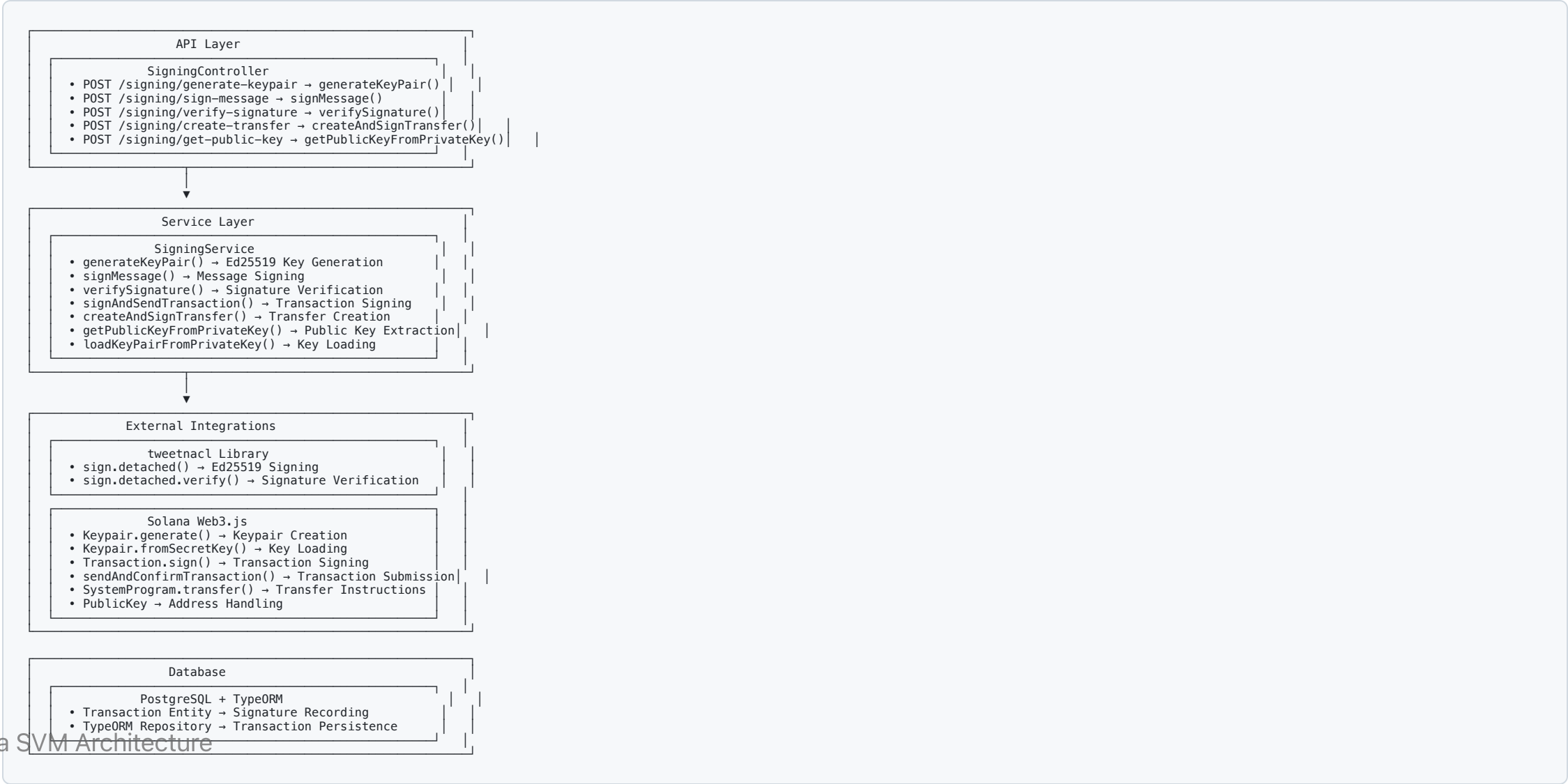
Ed25519 Digital Signatures

- **Algorithm:** Ed25519 elliptic curve cryptography
- **Key Size:** 256-bit keys (32 bytes)
- **Signature Size:** 512-bit signatures (64 bytes)
- **Security:** Post-quantum resistant design
- **Performance:** Fast signing and verification

Solana's Crypto Requirements

- **Transaction Signing:** All transactions must be signed
- **Account Security:** Private keys control account ownership
- **Program Security:** Signed deployments and upgrades
- **Multi-signature:** Multiple key validation

Architecture Overview



Key Management

Key Generation

```
generateKeyPair(): KeyPairResponse {  
  // Generate new Ed25519 keypair  
  const keypair = Keypair.generate();  
  
  // Return only public key (security best practice)  
  return {  
    publicKey: keypair.publicKey.toString(),  
    // private key is NEVER returned via API  
  };  
}
```

Private Key Handling

```
loadKeyPairFromPrivateKey(privateKeyBase64: string): Keypair {  
  try {  
    // Decode base64 private key
```

Message Signing

Signing Workflow

```
async signMessage(signMessageDto: SignMessageDto): Promise<SigningResult> {  
  try {  
    // Load keypair from private key  
    const keypair = this.loadKeyPairFromPrivateKey(signMessageDto.privateKey);  
  
    // Decode message from base64  
    const messageBytes = Buffer.from(signMessageDto.message, 'base64');  
  
    // Sign message using Ed25519  
    const signature = nacl.sign.detached(messageBytes, keypair.secretKey);  
  
    // Return base64 encoded signature  
    return {  
      signature: Buffer.from(signature).toString('base64'),  
      publicKey: keypair.publicKey.toString(),  
      success: true  
    };  
  } catch (error) {  
    throw new BadRequestException('Message signing failed');  
  }  
}
```

Signature Verification

Verification Process

```
async verifySignature(verifyDto: VerifySignatureDto): Promise<VerificationResult> {  
  try {  
    // Decode inputs from base64  
    const messageBytes = Buffer.from(verifyDto.message, 'base64');  
    const signatureBytes = Buffer.from(verifyDto.signature, 'base64');  
    const publicKey = new PublicKey(verifyDto.publicKey);  
  
    // Verify signature using Ed25519  
    const isValid = nacl.sign.detached.verify(  
      messageBytes,  
      signatureBytes,  
      publicKey.toBytes()  
    );  
  
    return {  
      isValid,  
      publicKey: publicKey.toString(),  
      message: isValid ? 'Signature is valid' : 'Signature is invalid'  
    };  
  } catch (error) {  
    throw new BadRequestException('Signature verification failed');
```

Transaction Signing

Complete Transaction Flow

```

async createAndSignTransfer(transferDto: CreateTransferDto): Promise<TransactionResult> {
  try {
    // Load sender's keypair
    const senderKeypair = this.loadKeyPairFromPrivateKey(transferDto.privateKey);

    // Create transfer instruction
    const transferInstruction = SystemProgram.transfer({
      fromPubkey: senderKeypair.publicKey,
      toPubkey: new PublicKey(transferDto.toAddress),
      lamports: transferDto.amount
    });

    // Build transaction
    const transaction = new Transaction().add(transferInstruction);

    // Get recent blockhash
    const { blockhash } = await this.connection.getRecentBlockhash();
    transaction.recentBlockhash = blockhash;
    transaction.feePayer = senderKeypair.publicKey;

    // Sign transaction
    transaction.sign(senderKeypair);

    // Submit to network
    const signature = await this.connection.sendAndConfirmTransaction(transaction);

    // Record transaction in database
    await this.recordTransaction(signature, transferDto);

    return {
      signature,
      success: true,
      transactionId: signature
    };
  } catch (error) {

```

Transaction Workflow

Signing Workflow Steps

1.  Load Keypair → Private Key to Keypair
2.  Build Transaction → Instruction Assembly
3.  Sign Transaction → Ed25519 Signing
4.  Submit to Network → `sendAndConfirmTransaction`
5.  Record Transaction → Database Persistence

Transaction Components

- **Instructions:** Program calls and data
- **Recent Blockhash:** Prevents replay attacks
- **Fee Payer:** Account paying transaction fees
- **Signatures:** Ed25519 signatures from required signers

API Endpoints

Key Management

- `POST /signing/generate-keypair` - Generate new Ed25519 keypair
- `POST /signing/get-public-key` - Extract public key from private key

Message Signing

- `POST /signing/sign-message` - Sign arbitrary message
- `POST /signing/verify-signature` - Verify message signature

Transaction Operations

- `POST /signing/create-transfer` - Create and sign SOL transfer

Response Formats

Security Considerations

Implemented Security Measures

- **Private Key Protection:** Never returned via API responses
- **Input Validation:** Strict format checking for all inputs
- **Error Sanitization:** No sensitive data in error messages
- **Base64 Encoding:** Safe transport of binary cryptographic data
- **Transaction Recording:** Complete audit trail

Cryptographic Best Practices

- **Ed25519 Standard:** Industry-standard elliptic curve cryptography
- **Deterministic Signatures:** Consistent signature generation
- **Replay Protection:** Blockhash prevents transaction replay
- **Multi-signature Ready:** Supports multiple signer validation

MPC Integration (Future)

Multi-Party Computation Features

- **Key Share Distribution:** Split private keys across multiple parties
- **Threshold Signing:** Require minimum signatures for validity
- **Secure Reconstruction:** Combine shares without exposing full key
- **Byzantine Fault Tolerance:** Resilient to malicious participants

MPC Benefits

- **Enhanced Security:** No single point of key compromise
- **Distributed Trust:** Multiple parties required for signing
- **Fault Tolerance:** System continues with some parties offline
- **Regulatory Compliance:** Addresses custody and control requirements

Key Takeaways

Cryptography Implementation

- **Ed25519 Focus:** Solana's native cryptographic algorithm
- **Secure Key Management:** Private keys never exposed via API
- **Complete Transaction Flow:** From key generation to network submission
- **Audit Trail:** All operations recorded in database

SVM Cryptographic Advantages

- **High Performance:** Fast Ed25519 operations at scale
- **Parallel Processing:** Multiple signature verifications simultaneously
- **Network Security:** Cryptographic validation of all state changes
- **Future MPC Ready:** Foundation for advanced cryptographic schemes